

Intern Portfolio Readiness Scorer

END-TO-END ENTERPRISE TECHNICAL DEPLOYMENT & BUSINESS INTELLIGENCE REPORT

Document Focus:	Comprehensive Engineering Milestone Report
GitHub Repository:	https://github.com/Shashikala-11/intern-portfolio-readiness-scorer
Target Corporate:	Graphura Private Limited, Engineering and Analytics Division
Project Team Lead:	Shashikala Gupta
Release Tracking:	May 25, 2026

1. Introduction

1.1 Project Context and Rationale

In modern software engineering organizations, evaluating the performance, operational capacity, and deployment readiness of technical interns is a critical path for permanent workforce integration. Traditionally, this validation relies heavily on subjective performance reviews, single-point final evaluations, and unverified summaries found on static resumes. This classic process leaves substantial room for human bias, lacks cross-organizational standards, and routinely fails to capture an engineer's dynamic coding trajectory.

At Graphura Private Limited, scaling technical internship frameworks demands a completely automated, data-driven approach. The Intern Portfolio Readiness Scorer project was initiated to close this operational gap. By shifting the evaluation framework away from self-declared resume blocks and directly toward real, verifiable source control profiles, the project maps continuous code metadata directly onto predictive business outcomes.

1.2 Scope and System Objectives

The operational boundaries of this implementation span across four core engineering pillars: real-time dataset harvesting from public APIs, non-linear feature transformation modeling, deep multi-variate statistical machine learning classification, and enterprise business intelligence visualization. The ultimate system objective is to instantly ingest an external profile locator, isolate high-impact code assets, parse documentation completeness, evaluate project velocity, and predict an integer employability metric coupled with a reliable operational placement recommendation.

Key Organizational Benchmarks:

- Scalability: Complete elimination of manual portfolio verification loops across thousands of quarterly entries.
- Objectivity: Processing based entirely on open-source contributions, code complexity factors, and live deployment verification layers.
- Predictive Intelligence: Training highly optimized production-grade classification boundaries capable of matching candidate features to organizational roles with a target accuracy exceeding 95 percent.

2. Problem Statement

2.1 The Friction of Subjective Screening

Human resources and engineering management branches face severe bottlenecks when evaluating talent. Portfolios vary widely across different technical stacks, formats, and design aesthetics. An evaluator looking over a candidate's repository history must manually review files, inspect the formatting of documentation structures, verify whether code functions correctly, and guess at the business value of the work. This qualitative methodology creates severe operational strain when scaled up to hundreds of incoming candidates.

2.2 Data Inconsistencies and Metric Isolation

Furthermore, raw numbers extracted directly from open-source profiles are frequently misleading when evaluated linearly. For example, a candidate might possess a large public repository count, but closer analysis reveals that these repositories are empty configurations or basic clones of tutorial repositories. Conversely, an exceptionally strong builder might possess only a few repositories, but each repository represents highly complex engineering work with multiple contributors, high fork counts, and continuous updates. Evaluating these candidates based purely on raw totals leads to poor talent assessment and high costs due to misaligned placements.

2.3 The Graphura Operational Mandate

Graphura requires a standardized, unified, and automated pipeline that completely removes human subjectivity from the primary screening loop. The specific engineering requirements demand a platform that handles:

Operational Failure Mode	Traditional Impact	Graphura Automation Standard
Resume Inflation	Unverified claims regarding tool usage and project impact go unchecked until late interview stages.	Direct validation via code metrics and repository configuration parsing.
Linear Profile Bias	Profiles are rated strictly on vanity metrics (e.g., total repositories) without analyzing depth.	Logarithmic scale transformation of code assets and multi-variate feature analysis.
Evaluation Latency	Manual code analysis limits screening speed to 3 to 4 candidates per engineer per hour.	Microservice execution capable of evaluating thousands of profiles in under 6 minutes.

3. Project Methodology

3.1 Scientific Approach and Research Design

The core research design follows an inductive data-mining methodology applied to open-source software engineering profiles. To model a reliable candidate evaluation engine, the pipeline must reflect real developer behaviors without relying on mock data. The mathematical foundation uses multi-variate vector normalization, where varied, scattered API counters are mapped into structured, continuous vectors between 1.0 and 10.0 based on an explicit grading rubric.

3.2 Data Sourcing and API Request Management

Data collection was handled entirely through direct integration with the GitHub REST API v3, pulling information from active candidate profiles across multiple global development hubs. To manage strict API limits, the connection logic was built to leverage authenticated token configurations, providing an allocation of 5,000 queries per hour. Rather than running slow, deeply nested loops to pull secondary code trees, the architecture uses structured parameter payloads to query optimized records in single-pass operations.

3.3 Engineering Target Categorization

The structured feature vector maps directly to three non-overlapping classification categories, which serve as the target labels for our machine learning classifiers:

- Job Ready (Score greater than or equal to 75): Candidates demonstrating sophisticated documentation standards, verified business code re-use, structured version histories, and multiple live deployments.
- Almost Ready (Score 50 to 74): Candidates possessing a solid technical codebase, but lacking across presentation elements or external documentation assets.
- Needs Improvement (Score less than 50): Candidates with low repo activity, minimal code footprints, or unoptimized documentation configurations.

4. System Architecture

4.1 Data Flow and Processing Topologies

The pipeline uses an asynchronous microservice topology designed for maximum throughput and isolation between the ingestion and classification steps. The application contains four core logical layers: the Ingestion Core, the Feature Engine, the Mathematical Transformation Array, and the Presentation Edge.

4.2 Component Block Schema

1. Ingestion Engine: Connects to public APIs and handles authentication, request limits, and payload validation. It captures name records, follower networks, repository counts, blog indicators, and project size metrics.
2. Feature Logic Matrix: Maps raw fields into structural parameters. It runs language-byte assessments to determine the candidate's exact technical track and tool distribution.
3. Mathematical Transformation Array: Ingests the raw values and converts them into the 1 to 10 scoring scale using non-linear normalization algorithms to remove outlier skewing.
4. Prediction and Service Edge: Serializes optimized model parameters to process incoming profile inputs, instantly returning scores and labels to both the Flask front-end and the enterprise Power BI dashboards.

Pipeline Interface Specifications:

The backend engine connects directly to a serialized LogisticRegression machine learning pipeline via a binary interface wrapper. When a user submits candidate profile parameters through the dashboard, the data flows instantly into the feature scaling matrix, evaluates the classification boundaries, and renders the predictions on the front-end interface in milliseconds.

5. Phase 1 & 2: Data Collection and Feature Engineering

5.1 Shared Data Extraction Execution

The data harvesting operation was executed cleanly by the data engineering track consisting of Shashikala Gupta, Ayush, and Shivangi. The sub-team built a high-speed data extraction engine to collect 100 percent real developer profiles without rate-limiting interruptions. By avoiding deep nested loops to check README directories on every single repository—which would create massive network slowdowns and run for over 20 hours—the engine pulls user repository catalogs exactly once per candidate, sorted by popularity using the repos endpoint. This approach successfully gathered 1,555 real rows of production code metadata in under 6 minutes.

5.2 Mathematical Feature Formulation

The raw metadata fields are transformed into normalized 1-to-10 scoring features using explicit algorithmic equations:

5.2.1 Business Relevance Score Evaluation

In open-source software, repository forks indicate that another engineer has actively copied a codebase to modify or build upon it, which represents a much stronger signal of real business utility than a casual repository star. To capture this relationship accurately, the script applies a weighted score where forks are multiplied by a factor of 2, combined with a natural logarithmic transformation to prevent popular baseline projects from skewing the model boundaries:

$$\text{Business Relevance} = \max(1.0, \min(10.0, \ln(1 + \text{Total Forks} * 2 + \text{Total Stars}) * 1.1 + 1.0))$$

5.2.2 Presentation and Documentation Proxy Evaluation

The presentation score analyzes the profile's layout and organization. Maintaining an active external blog link provides an initial score increase, while live project deployments via GitHub Pages add incremental points up to a maximum cap of 10.0. To evaluate documentation quality quickly and efficiently at scale, the engine uses average repository disk footprint size as a highly accurate proxy for content layout and documentation depth.

```
# Ingestion feature math snippet executed during pipeline pass
import numpy as np

biz_rel = np.clip(np.log1p(forks * 2 + stars) * 1.1 + 1.0, 1.0, 10.0)
pres_score = np.clip((has_blog * 4.0) + (gh_pages * 1.5) + 1.5, 1.0, 10.0)
```

```
doc_qual = np.clip(np.log1p(avg_size) * 0.9 + (has_blog * 1.5), 1.0, 10.0)
consistency = np.clip((public_repos * 0.15) + np.log1p(followers) * 0.4 + 2.0, 1.0, 10.0)
```

6. Phase 3: Predictive Modeling and Diagnostics

6.1 Machine Learning Implementation Framework

With the 1,555-record dataset completely structured and engineered, the modeling track trained three classification algorithms. The data split allocated 80 percent of the profiles for training and 20 percent for testing to ensure rigorous validation against unseen data.

6.2 Comparative Model Performance Analysis

The performance metrics and classification boundaries were carefully analyzed across all three frameworks to find the optimal fit for the live production microservice:

Model Architecture	Validation Accuracy	Precision (Weighted)	Recall (Weighted)	F1-Score (Weighted)
Logistic Regression (LR)	96.14%	0.96	0.96	0.96
Random Forest (RF)	94.86%	0.95	0.95	0.94
Decision Tree (DT)	94.21%	0.94	0.94	0.94

6.3 Justification for Selection of Primary Model

The Logistic Regression classifier was selected as the primary production engine for Graphura's live environment. While decision-tree frameworks performed well, they showed high variance across localized data splits and small accuracy drops due to strict value cuts. The Logistic Regression model creates highly stable probability boundaries across multi-class target labels, generalizes perfectly across varied roles, and features a lightweight footprint ideal for fast API microservices.

7. Phase 4: Production Flask Deployment and Power BI Analytics

7.1 Flask Microservice Implementation

The optimized Logistic Regression pipeline was serialized into production binaries using pickle and embedded into a high-performance Flask microservice web application. The application exposes an interactive user interface dashboard designed for human resources and engineering managers, allowing them to input any candidate's repository counters directly into the form fields.

The backend microservice intercepts the incoming request payload, passes the values through the mathematical feature scaling array, runs the data against the serialized classifier, and instantly returns the numerical score and employability label back to the web layout. This complete end-to-end automation replaces manual screening hours with sub-millisecond execution loops.

7.2 Enterprise Power BI Dashboard Engineering

To provide senior leadership with clear corporate visibility, the engineered dataset was integrated directly into an interactive Power BI dashboard environment. This dashboard processes the portfolio metrics to create cross-filtering visual reports, displaying distribution curves of candidate readiness across roles, tracking developer tools usage frequencies, and identifying placement bottlenecks. This application provides Graphura with macro-level talent insights alongside real-time individual screening capabilities.

System Operational Path:

Form Ingestion to Flask Backend Intercept to Feature Math Matrix to Serialized Model Scoring to Dashboard Update Rendering

8. Conclusion and Future Scope

8.1 Operational Project Conclusions

The successful development and rollout of the Intern Portfolio Readiness Scorer demonstrates that traditional, subjective talent screening can be replaced with an objective, data-driven software process. By leveraging real code metrics, log-scaled business relevance calculations, and verified deployment records from 1,555 live profiles, the project team built a highly reliable assessment pipeline. Achieving a peak validation accuracy of 96.14 percent ensures that candidate classifications match true portfolio capabilities, significantly reducing engineering review hours and placement mismatches.

8.2 Future Engineering Goals

The future roadmap for the system focuses on expanding its technical capabilities through three major milestones:

1. **Automated Source Code Analysis:** Integrating lightweight abstract syntax tree parsers to analyze code structure directly, measuring modularity, code reuse, and test coverage rather than relying on repository size metrics alone.
2. **Deep Multi-Platform Ingestion:** Expanding the data harvesters to pull data from other major platforms like GitLab, Bitbucket, and StackOverflow to build a unified developer evaluation hub.
3. **Neural Network Boundary Modeling:** Transitioning the predictive core to deep multi-layer perceptron architectures to discover subtler feature patterns as the database expands past tens of thousands of records.

9. Project Team Roles and Contribution Matrix

9.1 Strategic Leadership and Technical Execution Ledger

This enterprise technical milestone was successfully delivered through structured role allocation, individual engineering execution, and cross-functional collaboration across all project development tracks:

Name	Core Specialized Role	Explicit Project Contributions and Milestone Delivery
Shashikala Gupta	Project Team Lead and Ingestion Architecture Co-Lead	Managed the end-to-end engineering roadmap and checkpoints. Co-designed and optimized the high-speed data harvester script. Led the log-scaled feature matrix math configurations and coordinated the final microservice integrated product rollout.
Ayush & Shivangi	Data Engineering and Extraction Track	Collaborated on the core data harvesting script setup for the 1,555 rows. Partnered on defining evaluation matrix logic rules, parameter scales, and training data matrix preparation.
Vedant	Business Intelligence Track	Designed and built the enterprise Power BI dashboards and advanced business analytics reports. Modeled the metrics distributions, handled cross-filtering reports for management, and delivered actionable insights on candidate placement distributions.
Faraz	Machine Learning Track	Built, trained, and tuned the machine learning algorithms. Conducted cross-validation and hyperparameter optimization to achieve a peak validation accuracy of 96.14% with the Logistic Regression model, and built comparative baselines for the RF and DT models.
Saksham & Rajlakshmi	Full-Stack Deployment Track	Developed the production web app infrastructure. Wrapped the serialized model binaries into a scalable Flask microservice backend and designed the front-end user interface form layout to handle live queries seamlessly.

Thank You

We express our sincere gratitude to the engineering board, project mentors, and the leadership team at Graphura Private Limited for providing the operational guidance, resources, and deployment ecosystem necessary to bring this machine learning pipeline to life.

Graphura Private Limited • Confidential Corporate Release Version 1.0.0